

Embedded Systems mit RISC-V

6. Interrupts und Exceptions

V1.0, Patrick Ritschel



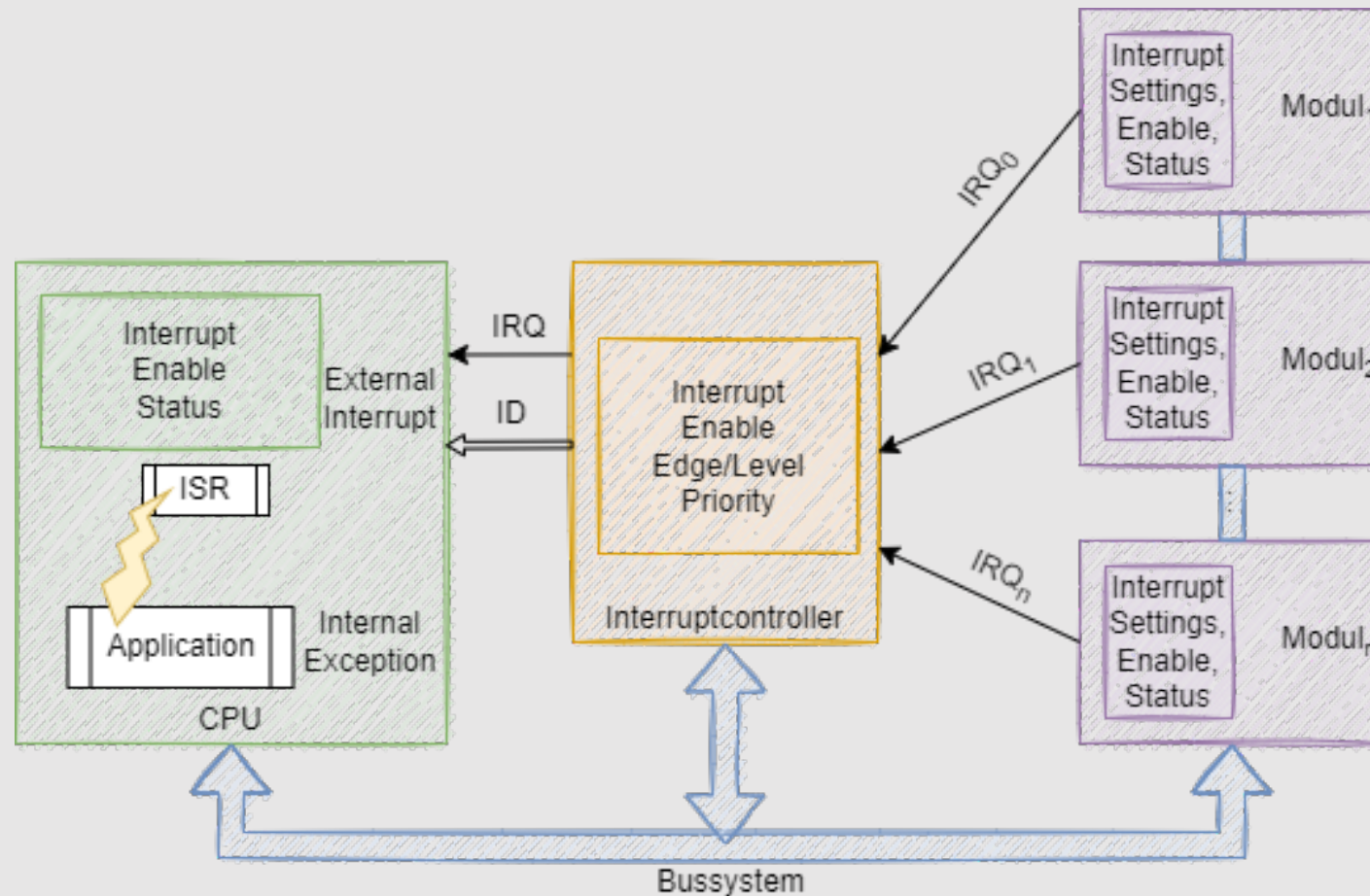
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {  
    float f;   
    f = value;  
    return f;  
}
```

Interrupt-Controller

Anbindung an das System



IIRFilter

Filter

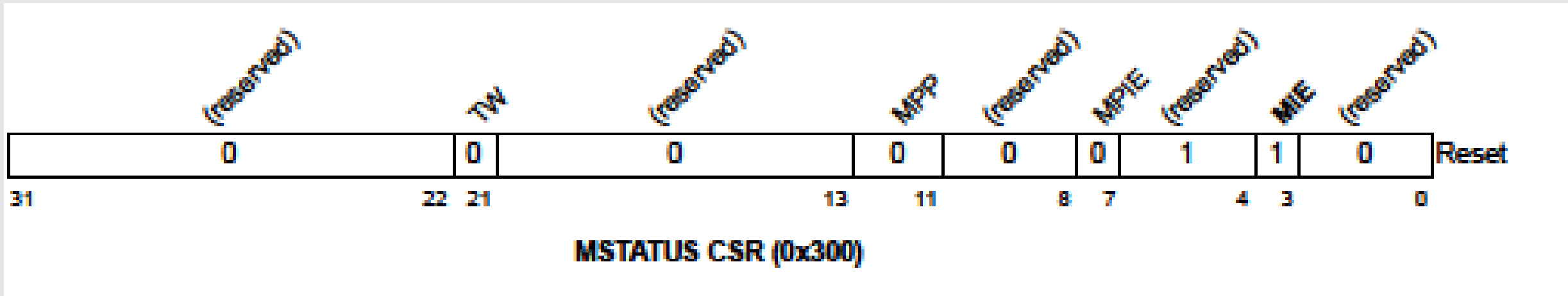
```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Exceptions und Interrupts

Arten und Interrupt Enable Status



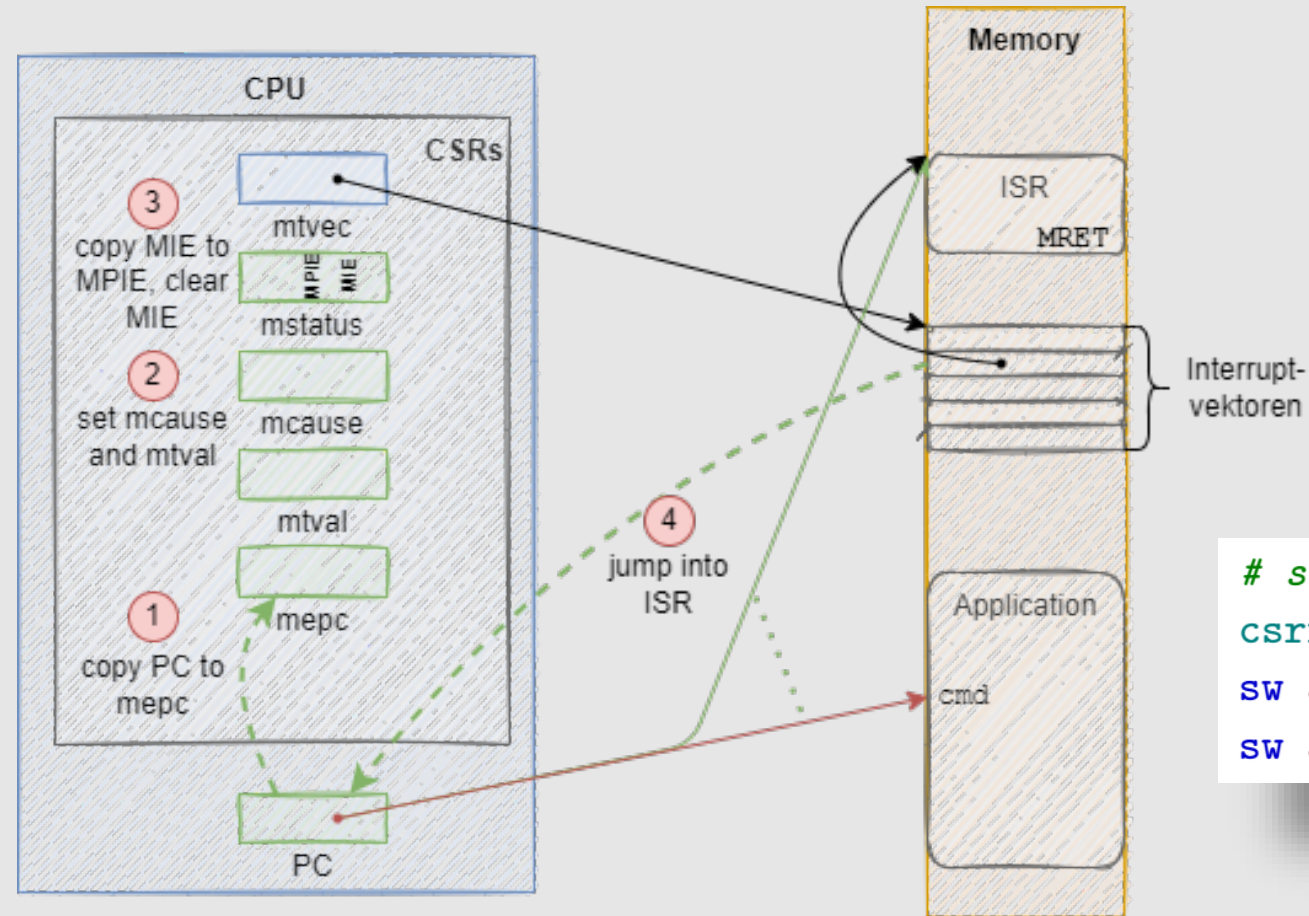
- Asynchronous (Non-Maskable / Maskable): Interrupts
- Synchronous (Precise / Imprecise): Exceptions



```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Interrupt Service Routine

Eintritt in die ISR



```
# save registers
csrrw a0, mscratch, a0 # exchange a0 and memory pointer
sw a1, 0(a0) # save a1
sw a2, 4(a0) # save a2
```

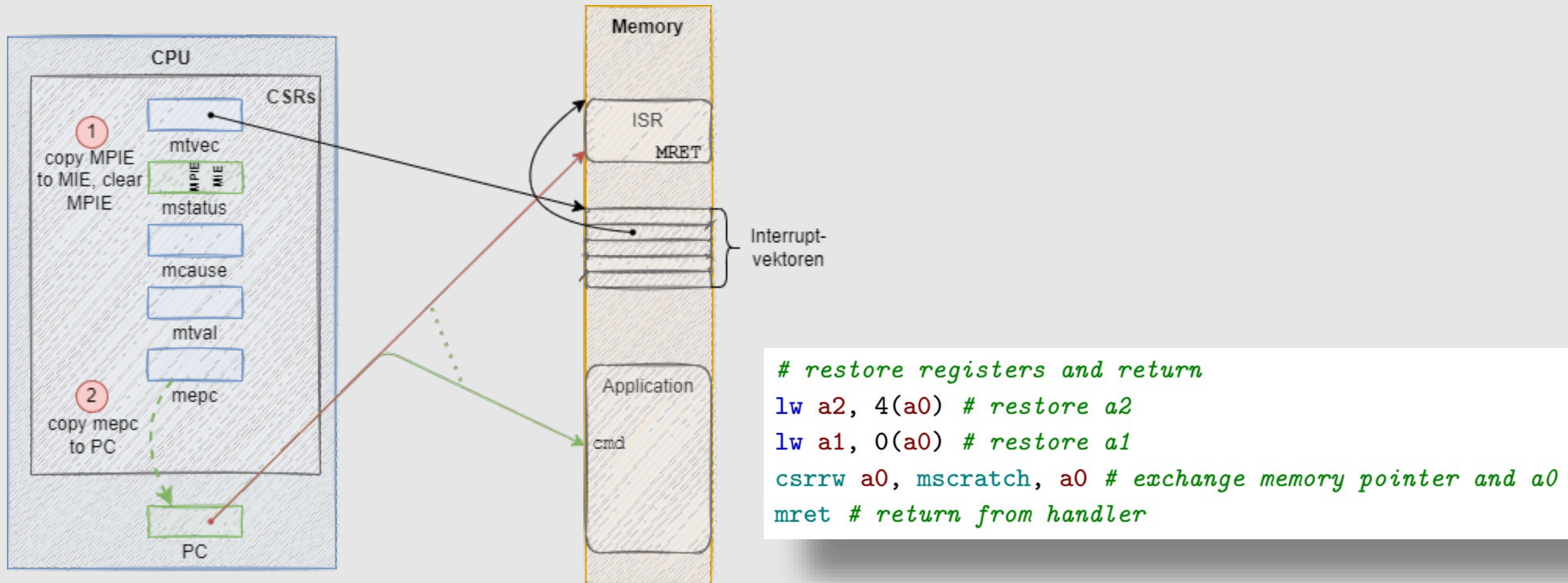
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```


Interrupt Service Routine

Rücksprung aus der ISR



IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Interrupts

Problematisches Unterbrechen



```
uint64_t getSysticks(void) {  
    return gSysticks;  
}
```

```
getSysticks:  
    lui a5,0x3fc8c  
    lw a0,784(a5) # 0x3fc8c310 <gSysticks>  
    lw a1,788(a5)  
    ret
```

```
// each ms in timer ISR  
gSysticks += 1;
```

```
uint64_t getSysticks(void) {  
    uint32_t intstate;  
    // save mstatus register in intstate and clear MIE  
    asm volatile (" csrrci %0, mstatus, 0x8" : "=r"(intstate));  
    // critical section  
    uint64_t systicks = gSysticks;  
    // restore mstatus from intstate  
    asm volatile (" csrw mstatus, %0" : : "r"(intstate));  
    return systicks;  
}
```

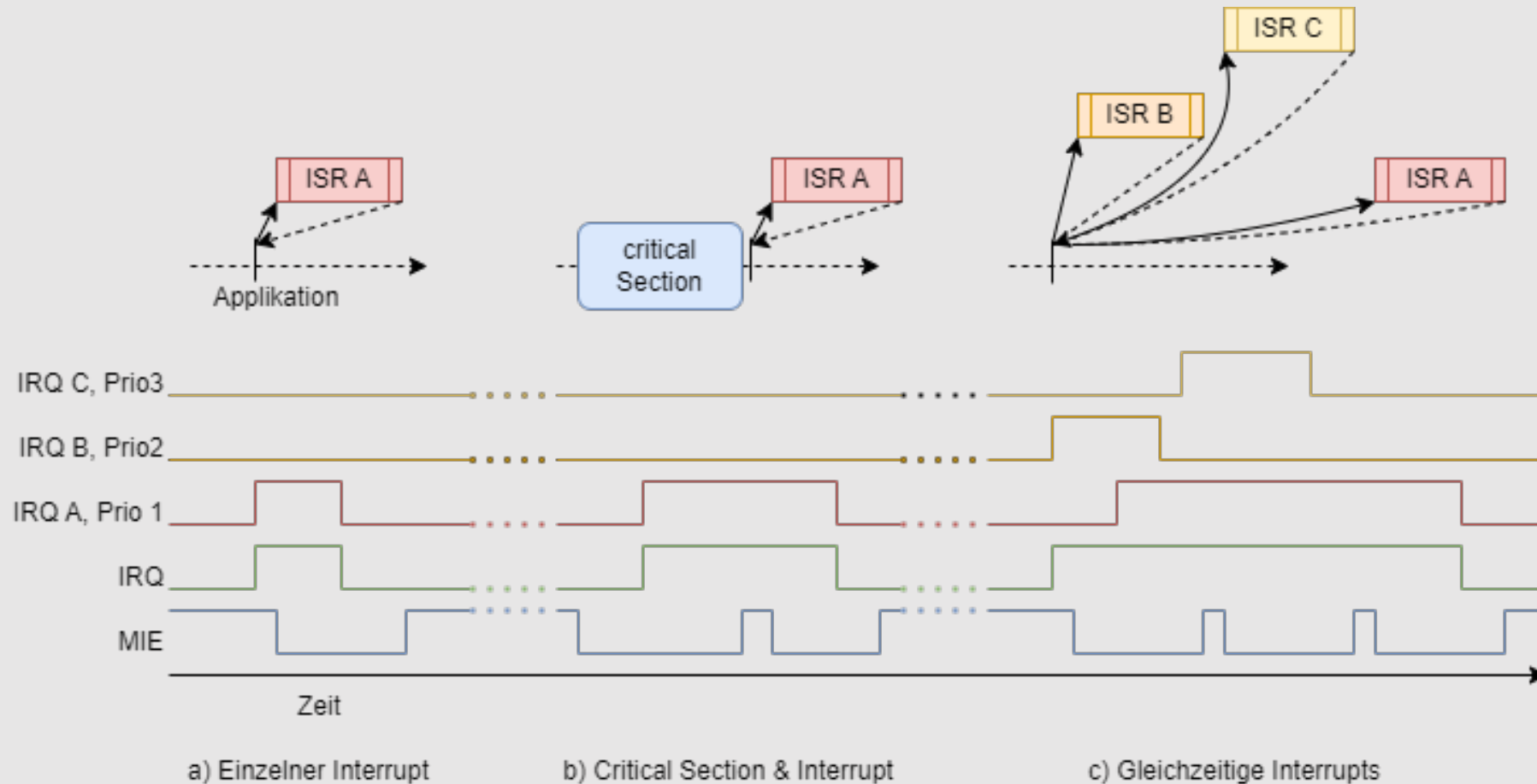
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {  
    [ ... ]  
}
```

Interrupt-Abarbeitung

Aufrufdarstellung und Signal-/Zeitverlauf



a) Einzelner Interrupt

b) Critical Section & Interrupt

c) Gleichzeitige Interrupts

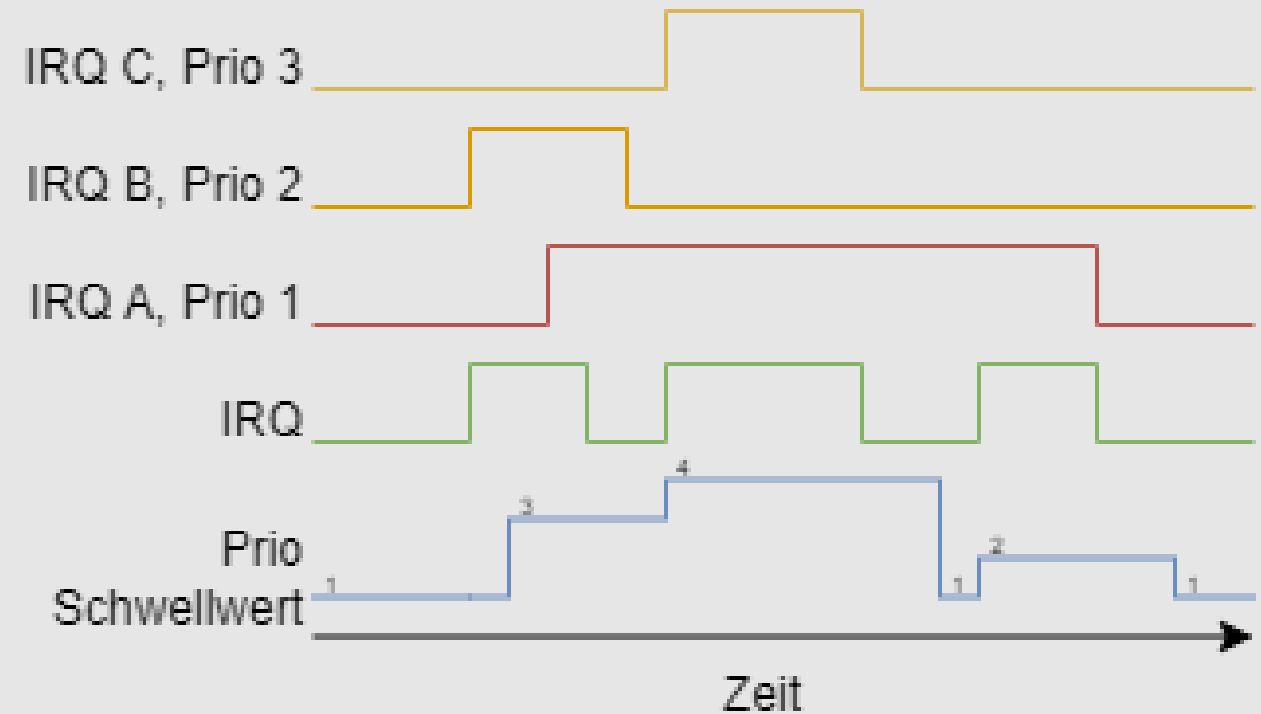
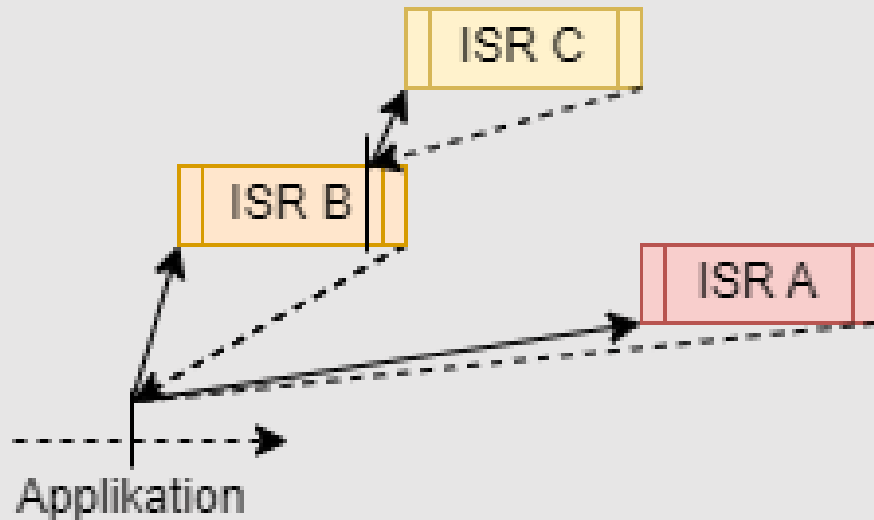
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

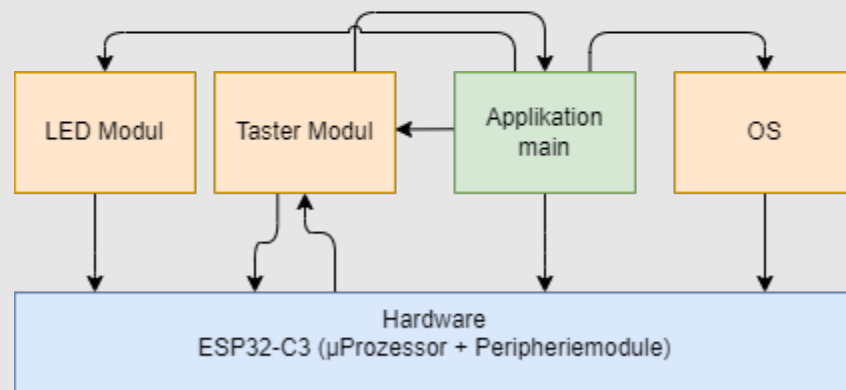
Interrupt-Abarbeitung

Verschachtelung

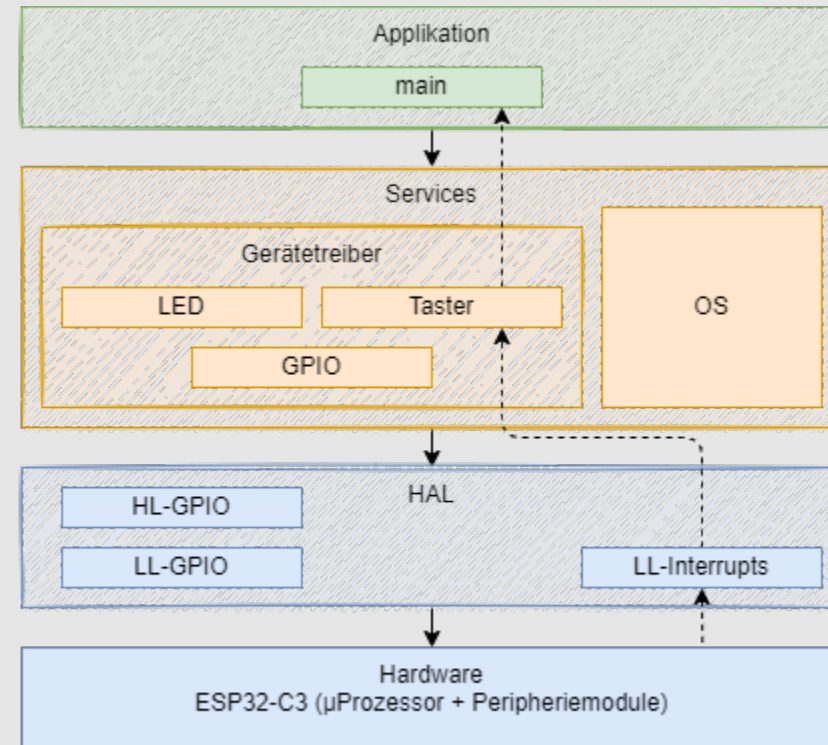


```
float iirfilter_filterValue(Filter* pFilter, float value) {
```


Schichtenarchitektur ohne und mit



a) ohne Schichtenarchitektur



b) mit Schichtenarchitektur

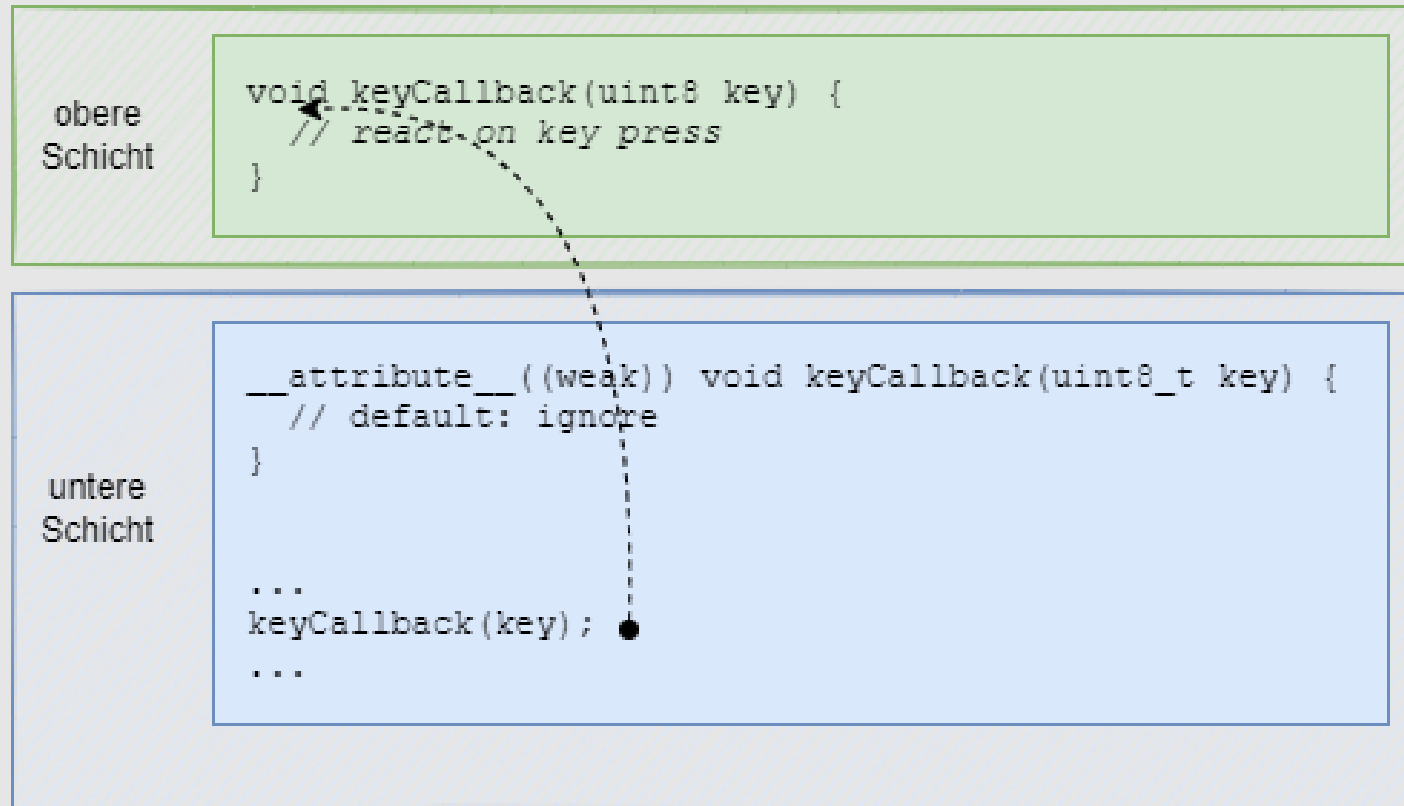
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Schichtenarchitektur

Statischer Callback



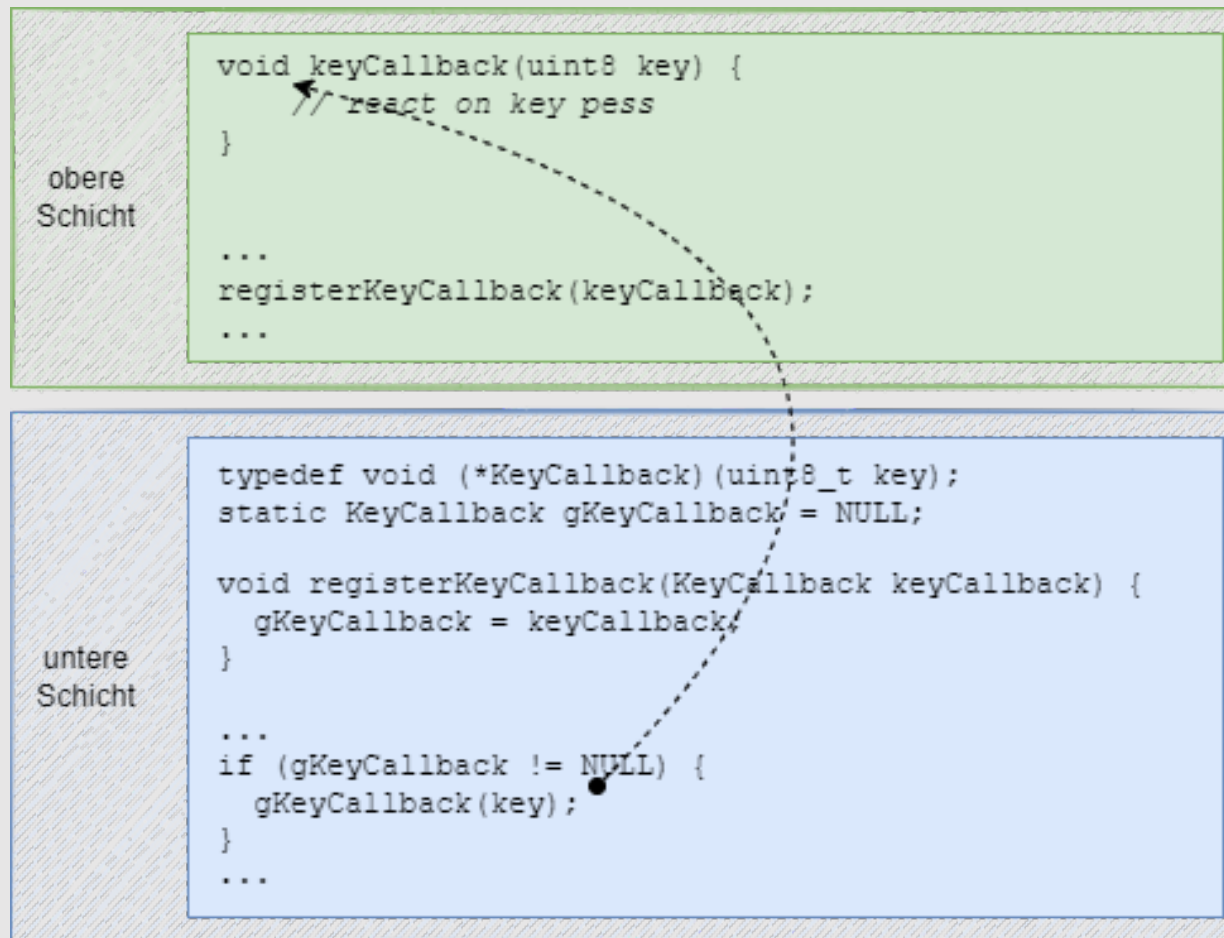
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {  
    [ ... ]  
}
```

Schichtenarchitektur

Dynamischer Callback



IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {  
    [ ... ]  
}
```

Interrupt bei Tastendruck ESP-IDF



```
.intr_type = GPIO_INTR_NEGEDGE
```

```
gpio_install_isr_service(0);
```

```
gpio_isr_handler_add(BTN_GPIO, btn_isr_handler, NULL);
```

```
static void IRAM_ATTR btn_isr_handler(void* arg) {  
    (void) arg;  
    gKeyPressed = true;  
}
```

```
while (true) {  
    if (gKeyPressed) {  
        ledLevel = !ledLevel;  
        gpio_set_level(LED1_GPIO, ledLevel);  
        gKeyPressed = false;  
    }  
    vTaskDelay(50 / portTICK_PERIOD_MS);  
}
```

IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```